

Solucionario Técnico: Bloque 2

Ficheros Binarios, Acceso Posicional Directo (RAF) y Serialización Segura

MÓDULO PROFESIONAL	CICLO Y UNIDAD	MODALIDAD / ENTORNO	DESTINATARIO / CARÁCTER
Acceso a Datos (0486)	DAM 2º Curso · UT1	Presencial / Distancia · Java 21	Docente · Solución Oficial Comentada

ALINEACIÓN CURRICULAR OFICIAL (REAL DECRETO 455/2010 · CURRÍCULO ASTURIAS)

RA1: Manejo de Ficheros

CE1.c (Ficheros Binarios y Acceso Aleatorio)

CE1.d (Serialización de Grafos de Objetos)

Solucionario oficial que resuelve la persistencia binaria tipada con `DataStreams` y `EOFException`, el direccionamiento físico por cálculo de offsets mediante `RandomAccessFile` (44 bytes/registro), y la eliminación de cabeceras redundantes en serialización incremental con `MiObjectOutputStream`.

Claves Técnicas de la Implementación:

- **Simetría en Tipos Primitivos:** El orden de lectura en `DataStream` debe coincidir exactamente con el de escritura (`writeInt` , `writeUTF` , `writeDouble` $\times 3$, `writeBoolean`).
- **Cierre de Bucle con `EOFException` :** Se implementa un `while(true)` intencionado, delegando en el bloque `catch (EOFException e)` la finalización ordenada de la lectura sin considerarlo un fallo.

Código Fuente Completo:

```

package soluciones_b2;

import java.io.*;

public class CajaNegraFlota {
    private static final String RUTA_FICHERO = "datos/flota_gps.dat";

    public static void registrarPosicion(int idVehiculo, String matricula, double lat, double lon, double vel, boolean enRuta) {
        File f = new File(RUTA_FICHERO);
        f.getParentFile().mkdirs();

        try (DataOutputStream dos = new DataOutputStream(
            new BufferedOutputStream(new FileOutputStream(f, true)))) {

            dos.writeInt(idVehiculo);
            dos.writeUTF(matricula);
            dos.writeDouble(lat);
            dos.writeDouble(lon);
            dos.writeDouble(vel);
            dos.writeBoolean(enRuta);

            System.out.printf("✅ Telemetría registrada: Vehículo %d (%s) - %.1f km/h%n", idVehiculo, matricula, vel);

        } catch (IOException e) {
            System.err.println("Error al escribir telemetría binaria: " + e.getMessage());
        }
    }

    public static void generarInformeFlota() {
        File f = new File(RUTA_FICHERO);
        if (!f.exists()) {
            System.out.println("No hay datos de telemetría registrados.");
            return;
        }

        int totalRegistros = 0;
        int excesosVelocidad = 0;

        System.out.println("=====");
        System.out.println("                AUDITORÍA DE FLOTA: EXCESOS DE VELOCIDAD (>120 km/h) ");
        System.out.println("=====");

        try (DataInputStream dis = new DataInputStream(
            new BufferedInputStream(new FileInputStream(f)))) {

            while (true) {
                int id = dis.readInt();
                String mat = dis.readUTF();
                double lat = dis.readDouble();
                double lon = dis.readDouble();
                double vel = dis.readDouble();
                boolean enRuta = dis.readBoolean();
                totalRegistros++;

                if (vel > 120.0) {
                    excesosVelocidad++;
                    System.out.printf("⚠️ ALERTA: Vehículo %d [%s] a %.2f km/h en pos (%.4f, %.4f)\n",
                        id, mat, vel, lat, lon, enRuta);
                }
            }
        }
    }
}

```

```
    } catch (EOFException e) {
        System.out.println("-----");
        System.out.printf("Fin de informe: %d tramas procesadas (%d infracciones detectadas).%n",
            totalRegistros, excesosVelocidad);
    } catch (IOException e) {
        System.err.println("Error leyendo fichero binario: " + e.getMessage());
    }
}

public static void main(String[] args) {
    registrarPosicion(1, "1234-BBB", 43.556, -5.924, 115.0, true);
    registrarPosicion(2, "5678-CCC", 43.535, -5.661, 132.5, true);
    registrarPosicion(3, "9012-DDD", 43.361, -5.849, 90.0, false);
    registrarPosicion(2, "5678-CCC", 43.540, -5.670, 128.0, true);

    generarInformeFlota();
}
}
```

Claves Técnicas de la Implementación:

- **Longitud Fija e Invariabilidad:** Los Strings se escriben con `writeChars()` tras rellenar con espacios o truncar mediante `ajustarTexto()`.
- **Salto Quirúrgico de Offset:** `raf.seek((pos * 44) + 40)` salta exactamente a los 4 bytes del campo `stock`, permitiendo actualizarlo sin tocar el ID ni los textos.
- **Truncado Físico en Disco:** `raf.setLength(maxRegistros * 44)` reduce instantáneamente el tamaño del fichero en disco.

Código Fuente Completo:

```

package soluciones_b2;

import java.io.File;
import java.io.IOException;
import java.io.RandomAccessFile;

public class AlmacenRAF {
    // 4B (id) + 16B (8 chars) + 20B (10 chars) + 4B (stock) = 44 Bytes
    public static final int TAM_REGISTRO = 44;
    private final File archivo;

    public AlmacenRAF(String ruta) {
        this.archivo = new File(ruta);
        this.archivo.getParentFile().mkdirs();
    }

    private String ajustarTexto(String texto, int longitudFija) {
        if (texto == null) texto = "";
        if (texto.length() > longitudFija) {
            return texto.substring(0, longitudFija);
        }
        StringBuilder sb = new StringBuilder(texto);
        while (sb.length() < longitudFija) {
            sb.append(' ');
        }
        return sb.toString();
    }

    public void insertarArticulo(int posicion, int id, String codigo, String nombre, int stock) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            long offset = (long) posicion * TAM_REGISTRO;
            raf.seek(offset);

            raf.writeInt(id);
            raf.writeChars(ajustarTexto(codigo, 8)); // 8 * 2 = 16 bytes
            raf.writeChars(ajustarTexto(nombre, 10)); // 10 * 2 = 20 bytes
            raf.writeInt(stock); // 4 bytes

            System.out.printf("✅ Artículo insertado en posición %d (offset: %d bytes)\n", posicion, offset);
        }
    }

    public void actualizarStock(int posicion, int nuevoStock) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
            long offsetStock = ((long) posicion * TAM_REGISTRO) + 40;
            if (offsetStock + 4 > raf.length()) {
                throw new IndexOutOfBoundsException("La posición " + posicion + " no existe en el archivo");
            }

            raf.seek(offsetStock);
            raf.writeInt(nuevoStock);

            System.out.printf("🔄 Stock actualizado en disco para pos %d -> Nuevo stock: %d\n", posicion, nuevoStock);
        }
    }

    public void consultarArticulo(int posicion) throws IOException {
        try (RandomAccessFile raf = new RandomAccessFile(archivo, "r")) {
            long offset = (long) posicion * TAM_REGISTRO;
            if (offset + TAM_REGISTRO > raf.length()) {
                throw new IndexOutOfBoundsException("La posición " + posicion + " no existe en el archivo");
            }
        }
    }
}

```

```

        System.out.println("❌ No existe registro en la posición " + posicion);
        return;
    }

    raf.seek(offset);
    int id = raf.readInt();

    char[] cCod = new char[8];
    for (int i = 0; i < 8; i++) cCod[i] = raf.readChar();
    String codigo = new String(cCod).trim();

    char[] cNom = new char[10];
    for (int i = 0; i < 10; i++) cNom[i] = raf.readChar();
    String nombre = new String(cNom).trim();

    int stock = raf.readInt();

    System.out.printf("📦 [Pos %d] ID: %d | Código: %s | Nombre: %s | Stock: %d uds.%n",
        posicion, id, codigo, nombre, stock);
    }
}

public long obtenerTotalArticulos() throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(archivo, "r")) {
        return raf.length() / TAM_REGISTRO;
    }
}

public void truncarAlmacen(int maxRegistros) throws IOException {
    try (RandomAccessFile raf = new RandomAccessFile(archivo, "rw")) {
        long nuevaLongitud = (long) maxRegistros * TAM_REGISTRO;
        raf.setLength(nuevaLongitud);
        System.out.printf("✂️ Archivo truncado a %d registros (%d bytes).%n", maxRegistros, nuevaLongitud);
    }
}

public static void main(String[] args) throws IOException {
    AlmacenRAF almacen = new AlmacenRAF("datos/articulos.dat");

    almacen.insertarArticulo(0, 101, "COD-0001", "Monitor 4K", 15);
    almacen.insertarArticulo(1, 102, "COD-0002", "Teclado Mec", 50);
    almacen.insertarArticulo(2, 103, "COD-0003", "Mouse Pad", 100);

    System.out.println("Antes de modificar:");
    almacen.consultarArticulo(1);

    almacen.actualizarStock(1, 42);

    System.out.println("Después de modificar:");
    almacen.consultarArticulo(1);
}
}

```

Solución Ejercicio 3: Auditoría Bancaria con Append Seguro y transient

(AuditoriaBancaria.java)

NIVEL
AVANZADO

Claves Técnicas de la Implementación:

- **Subclase `MiObjectOutputStream`** : Sobrescribe `writeStreamHeader()` vaciándolo para anular los 4 bytes de cabecera mágica (`0xACED`) que corromperían la lectura incremental.
- **Protección de Información Sensible con `transient`** : `private transient String pinAutorizacion` garantiza que el PIN vive en memoria durante la sesión pero se recupera como `null` en disco.
- **Patrón Factory / Operador Ternario**: Comprueba `archivo.exists() && archivo.length() > 0` para alternar limpiamente entre `ObjectOutputStream` estándar y `MiObjectOutputStream` .

Código Fuente Completo:

```

package soluciones_b2;

import java.io.*;
import java.time.LocalDateTime;

public class AuditoriaBancaria {

    // 1. Subclase que anula la escritura de cabecera en modo append
    public static class MiObjectOutputStream extends ObjectOutputStream {
        public MiObjectOutputStream(OutputStream out) throws IOException {
            super(out);
        }

        @Override
        protected void writeStreamHeader() throws IOException {
            // No escribe cabecera secundaria intermedia (0xACED0005)
        }
    }

    // 2. Entidad Serializable con transient
    public static class Transaccion implements Serializable {
        private static final long serialVersionUID = 1L;

        private long id;
        private String cuentaOrigen;
        private String cuentaDestino;
        private double importe;
        private LocalDateTime fecha;
        private transient String pinAutorizacion; // ¡NO se persiste!

        public Transaccion(long id, String origen, String destino, double importe, String pin) {
            this.id = id;
            this.cuentaOrigen = origen;
            this.cuentaDestino = destino;
            this.importe = importe;
            this.fecha = LocalDateTime.now();
            this.pinAutorizacion = pin;
        }

        @Override
        public String toString() {
            return String.format("[%s] ID: %d | %s -> %s | %.2f € | PIN RAM: %s",
                fecha.toLocalTime(), id, cuentaOrigen, cuentaDestino, importe, pinAutorizacion);
        }
    }

    // 3. Método de registro con discriminación automática de cabecera
    public static void registrarOperacion(File archivo, Transaccion t) {
        archivo.getParentFile().mkdirs();
        boolean yaExisteConDatos = archivo.exists() && archivo.length() > 0;

        try (FileOutputStream fos = new FileOutputStream(archivo, true);
            ObjectOutputStream oos = yaExisteConDatos
                ? new MiObjectOutputStream(fos)
                : new ObjectOutputStream(fos)) {

            oos.writeObject(t);
            oos.flush();
            System.out.println("✅ Transacción registrada: ID " + t.id);
        }
    }
}

```

```

    } catch (IOException e) {
        System.err.println("Error al serializar transacción: " + e.getMessage());
    }
}

// 4. Lectura secuencial completa hasta EOFException
public static void listarHistorial(File archivo) {
    if (!archivo.exists()) {
        System.out.println("No existe archivo de auditoría.");
        return;
    }

    System.out.println("=====");
    System.out.println("                HISTORIAL DE AUDITORÍA BANCARIA (DESERIALIZACIÓN)                ");
    System.out.println("=====");

    int contador = 0;
    try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream(archivo))) {
        while (true) {
            Transaccion t = (Transaccion) ois.readObject();
            contador++;
            System.out.println("  " + t);
        }
    } catch (EOFException e) {
        System.out.println("-----");
        System.out.printf("Auditoría finalizada: %d transacciones recuperadas sin corrupción.%n", contador);
    } catch (ClassNotFoundException e) {
        System.err.println("Error: Clase Transaccion no encontrada.");
    } catch (StreamCorruptedException e) {
        System.err.println("✖ ERROR GRAVE: Stream corrupto por doble cabecera: " + e.getMessage());
    } catch (IOException e) {
        System.err.println("Error de lectura: " + e.getMessage());
    }
}

public static void main(String[] args) {
    File f = new File("datos/transacciones.ser");
    f.delete(); // Limpiar pruebas anteriores

    registrarOperacion(f, new Transaccion(1001, "ES10-0001", "ES10-0002", 350.00, "1234"));
    registrarOperacion(f, new Transaccion(1002, "ES10-0003", "ES10-0001", 1200.50, "9999"));
    registrarOperacion(f, new Transaccion(1003, "ES10-0002", "ES10-0005", 75.25, "0000"));

    listarHistorial(f);
}
}

```